Academy Hub — Sesión 6

Imágenes múltiples, favoritos, búsqueda, filtros, paginación y consultas ORM

> **Nota:** esta sesión aplica el patrón del PDF de la Sesión 6 (múltiples imágenes, favoritos, buscador, filtros, ordenación, paginación, contador de visitas y optimización ORM) a todas las apps de Academy Hub, respetando su dominio real: cursos, academias, profesores, convocatorias, inscripciones, etc.

Índice

1. Objetivo general
2. Qué vamos a aprender
3. Flujo que queremos conseguir
4. Parte común: infraestructura y convenciones
5. `courses` — Curso como entidad central
6. `academies` — Academia como entidad pública
7. `reviews` — Reseñas con filtros y paginación
8. `level_tests` — Pruebas de nivel con filtros
9. `accounts` — Listado de usuarios
10. `enrollments` — Listado con filtros por estado
11. `payments` — Listado con filtros por estado
12. `notifications` — Paginación y filtros
13. `evaluations` — Listado con paginación
14. `talent` — Competencias con filtros
15. `search` — Buscador unificado
16. `core` — ContactMessage con filtros
17. `dashboard` — Contadores y métricas
18. Optimización ORM transversal
19. Paginación con querystring
20. Tests transversales
21. Checklist global
22. Registro de sesión
23. Commit recomendado
24. Resultado final

---

1. Objetivo general

En la Sesión 5 conseguimos que Academy Hub tenga CRUD completo en todas las apps. Ahora vamos a hacer que la plataforma sea **navegable**:

- Subida de varias imágenes por curso y por academia.
- Favoritos de cursos y de academias.
- Búsqueda por texto con `Q`.
- Filtros por categoría, nivel, modalidad, academia, profesor y rango de precio.
- Ordenación.
- Paginación con querystring persistente.
- Contador de visitas con `F()`.
- Optimización ORM con `select_related`, `prefetch_related` y `distinct()`.
- Tests exhaustivos de todo lo anterior.

Al terminar, Academy Hub se comportará como una plataforma real.

---

2. Qué vamos a aprender

- `request.FILES`
- Subida múltiple de archivos
- `getlist()`
- Objetos `Q` (OR / AND)
- Expresiones `F`
- Query strings y parámetros GET
- `filter()`, `exclude()`, `order_by()`
- `distinct()`
- `exists()`
- `count()`
- `get_or_create()`
- Favoritos como modelo intermedio
- `UniqueConstraint`
- Paginación con persistencia de filtros
- `select_related` y `prefetch_related`
- Contador de visitas
- Tests de búsqueda, filtros, favoritos y contador

---

3. Flujo que queremos conseguir

Un visitante podrá entrar en:

```
/courses/
```

y hacer cosas como:

- Buscar: `"inglés"`
- Filtrar:
  - Categoría: Idiomas
  - Nivel: Intermedio
  - Modalidad: Online
  - Academia: English Now
  - Precio: 200 € - 600 €
- Ordenar por:
  - Más recientes
  - Precio ascendente
  - Precio descendente
  - Más vistos

Resultado:

```
Inglés B2 intensivo — English Now — 480 €
Inglés B1 semipresencial — English Now — 320 €
```

También podrá:

- Añadir un curso a favoritos.
- Añadir una academia a favoritos.
- Consultar sus favoritos.

---

4. Parte común: infraestructura y convenciones

4.1. Multipart en formularios con archivos

Todos los formularios que acepten imágenes deben llevar:

```
enctype="multipart/form-data"
```

Sin esto, `request.FILES` estará vacío.

4.2. Campo múltiple reutilizable

Creamos un mixin de campo múltiple en `core/forms.py`:

`core/forms.py`

```
from django import forms


class MultipleFileInput(forms.ClearableFileInput):
    allow_multiple_selected = True
```

```
class MultipleFileField(forms.ImageField):
    def __init__(self, *args, **kwargs):
        kwargs.setdefault("widget", MultipleFileInput())
        super().__init__(*args, **kwargs)

    def clean(self, data, initial=None):
        single_file_clean = super().clean
        if isinstance(data, (list, tuple)):
            return [
                cleaned_file
                for item in data
                if (cleaned_file := single_file_clean(item, initial))
            ]
        if data in self.empty_values:
            return []
        return [single_file_clean(data, initial)]
```

Este campo se usa en `courses/forms.py` y `academies/forms.py`.

4.3. Validaciones de imágenes

Se añaden dos validaciones comunes a los formularios con imágenes:

- Máximo 8 imágenes.
- Máximo 5 MB por imagen.

4.4. Contador de visitas con F()

Se implementa con `F("views_count") + 1` para evitar condiciones de carrera.

4.5. Paginación con querystring persistente

Cada `ListView` con filtros pasa a `get_context_data`:

```
context["querystring"] = self.request.GET.copy().urlencode()
```

Y en el template:

```
<a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">Siguiente</a>
```

4.6. Optimización ORM

Reglas:

- `select_related` para `ForeignKey` y `OneToOne`.
- `prefetch_related` para `OneToMany` y `ManyToMany`.

- `distinct()` cuando se aplican filtros sobre relaciones `1:N`.

4.7. Modelo Favorite por entidad

Dos modelos separados:

- `CourseFavorite` en `courses/models.py`.
- `AcademyFavorite` en `academies/models.py`.

Ambos con `UniqueConstraint` por `(user, entidad)`.

---

5. `courses` — Curso como entidad central

5.1. Modelo `CourseFavorite`

`courses/models.py`

```
class CourseFavorite(models.Model):
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="course_favorites",
    )
    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
        related_name="favorites",
    )
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        constraints = [
            models.UniqueConstraint(
                fields=["user", "course"],
                name="unique_user_course_favorite",
            ),
        ]

    def __str__(self):
        return f"{self.user} - {self.course}"
```

5.2. Añadir `views_count` a `Course`

Verifica que `Course` ya tiene `views_count`. Si no, añádelo:

```
views_count = models.PositiveIntegerField(default=0)
```

5.3. Formulario con imágenes múltiples

courses/forms.py

```
from django import forms

from core.forms import MultipleFileField

from .models import Category, Convocatoria, Course


class CourseForm(forms.ModelForm):
    images = MultipleFileField(required=False)

    class Meta:
        model = Course
        fields = [
            "academy",
            "category",
            "teacher",
            "title",
            "description",
            "objectives",
            "competences",
            "requirements",
            "level",
            "modality",
            "enrollment_mode",
            "requires_level_test",
            "is_negotiable",
            "duration_hours",
        ]

    def __init__(self, *args, **kwargs):
        user = kwargs.pop("user", None)
        super().__init__(*args, **kwargs)
        if user is not None:
            self.fields["academy"].queryset = user.academies.filter(is_active=True)
            self.fields["teacher"].queryset = self.fields["teacher"].queryset.filter(
                academy__owner=user,
                is_active=True,
            )

    def clean_images(self):
        images = self.cleaned_data.get("images") or []

        if len(images) > 8:
            raise forms.ValidationError(
                "Puedes subir como máximo 8 imágenes."
            )

        max_size = 5 * 1024 * 1024  # 5 MB
        for image in images:
```

```
            if image.size > max_size:
                raise forms.ValidationError(
                    "Cada imagen no puede superar los 5 MB."
                )

        return images
```

5.4. `CourseCreateView` con imágenes

courses/views.py

```
from django.db import transaction
from django.db.models import F, Q
from django.shortcuts import get_object_or_404, redirect
from django.views.decorators.http import require_POST
from django.contrib.auth.decorators import login_required
from django.utils.http import url_has_allowed_host_and_scheme

from .models import (
    Category,
    Convocatoria,
    Course,
    CourseFavorite,
    CourseImage,
    CourseLevel,
    CourseModality,
    CourseStatus,
)


class CourseListView(ListView):
    model = Course
    template_name = "courses/course_list.html"
    context_object_name = "courses"
    paginate_by = 12

    def get_queryset(self):
        queryset = (
            Course.objects
            .filter(status=CourseStatus.PUBLISHED)
            .select_related("academy", "category", "teacher")
            .prefetch_related("images", "convocatorias")
        )

        query = self.request.GET.get("q", "").strip()
        if query:
            queryset = queryset.filter(
                Q(title__icontains=query)
                | Q(description__icontains=query)
                | Q(objectives__icontains=query)
                | Q(competences__icontains=query)
                | Q(academy__name__icontains=query)
```

```
                | Q(category__name__icontains=query)
            )

        category = self.request.GET.get("category")
        if category:
            queryset = queryset.filter(category__slug=category)

        academy = self.request.GET.get("academy")
        if academy:
            queryset = queryset.filter(academy__slug=academy)

        level = self.request.GET.get("level")
        if level and level in CourseLevel.values:
            queryset = queryset.filter(level=level)

        modality = self.request.GET.get("modality")
        if modality and modality in CourseModality.values:
            queryset = queryset.filter(modality=modality)

        min_price = self._parse_decimal("min_price")
        max_price = self._parse_decimal("max_price")

        if min_price is not None:
            queryset = queryset.filter(
                convocatorias__price__gte=min_price,
                convocatorias__is_active=True,
            )
        if max_price is not None:
            queryset = queryset.filter(
                convocatorias__price__lte=max_price,
                convocatorias__is_active=True,
            )

        ordering_map = {
            "newest": "-created_at",
            "oldest": "created_at",
            "price_asc": "min_price",
            "price_desc": "-min_price",
            "popular": "-views_count",
        }
        ordering = self.request.GET.get("ordering", "newest")
        queryset = queryset.order_by(ordering_map.get(ordering, "-created_at"))

        return queryset.distinct()

    def _parse_decimal(self, key):
        from decimal import Decimal, InvalidOperation
        raw = self.request.GET.get(key)
        if not raw:
            return None
        try:
            return Decimal(raw)
        except (InvalidOperation, ValueError):
```

```
            return None

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["categories"] = Category.objects.filter(is_active=True)
        context["levels"] = CourseLevel.choices
        context["modalities"] = CourseModality.choices

        context["favorite_course_ids"] = set()
        if self.request.user.is_authenticated:
            context["favorite_course_ids"] = set(
                CourseFavorite.objects
                .filter(user=self.request.user)
                .values_list("course_id", flat=True)
            )

        query_params = self.request.GET.copy()
        query_params.pop("page", None)
        context["querystring"] = query_params.urlencode()

        return context


class CourseCreateView(LoginRequiredMixin, UserPassesTestMixin, CreateView):
    model = Course
    form_class = CourseForm
    template_name = "courses/course_form.html"

    def test_func(self):
        return (
            self.request.user.is_academy
            and self.request.user.academies.exists()
        )

    def get_form_kwargs(self):
        kwargs = super().get_form_kwargs()
        kwargs["user"] = self.request.user
        return kwargs

    def form_valid(self, form):
        form.instance.status = CourseStatus.PUBLISHED

        with transaction.atomic():
            response = super().form_valid(form)

            images = form.cleaned_data.get("images") or []
            for position, image in enumerate(images):
                CourseImage.objects.create(
                    course=self.object,
                    image=image,
                    is_primary=(position == 0),
                    position=position,
                )
```

```
        messages.success(self.request, "Curso publicado correctamente.")
        logger.info(
            "Course created: id=%s user=%s",
            self.object.pk,
            self.request.user.pk,
        )
        return response


@login_required
@require_POST
def toggle_course_favorite(request, pk):
    course = get_object_or_404(
        Course,
        pk=pk,
        status=CourseStatus.PUBLISHED,
    )

    next_url = request.POST.get("next", "")
    if not url_has_allowed_host_and_scheme(
        next_url,
        allowed_hosts={request.get_host()},
        require_https=request.is_secure(),
    ):
        next_url = course.get_absolute_url()

    if course.academy.owner_id == request.user.pk:
        messages.warning(
            request,
            "No puedes guardar tu propio curso.",
        )
        return redirect(next_url)

    with transaction.atomic():
        favorite, created = CourseFavorite.objects.get_or_create(
            user=request.user,
            course=course,
        )
        if not created:
            favorite.delete()
            messages.info(request, "Curso eliminado de favoritos.")
        else:
            messages.success(request, "Curso añadido a favoritos.")

    return redirect(next_url)


class CourseFavoriteListView(LoginRequiredMixin, ListView):
    model = CourseFavorite
    template_name = "courses/course_favorite_list.html"
    context_object_name = "favorites"
    paginate_by = 12
```

```
    def get_queryset(self):
        return (
            CourseFavorite.objects
            .filter(user=self.request.user)
            .select_related(
                "course",
                "course__academy",
                "course__category",
                "course__teacher",
            )
            .prefetch_related("course__images")
        )


class CourseDetailView(DetailView):
    model = Course
    template_name = "courses/course_detail.html"
    context_object_name = "course"

    def get_queryset(self):
        return (
            Course.objects
            .select_related("academy", "category", "teacher")
            .prefetch_related("images", "convocatorias")
        )

    def get_object(self, queryset=None):
        queryset = queryset or self.get_queryset()
        course = queryset.get(pk=self.kwargs["pk"], slug=self.kwargs["slug"])

        Course.objects.filter(pk=course.pk).update(
            views_count=F("views_count") + 1
        )
        course.refresh_from_db(fields=["views_count"])
        return course

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["is_favorite"] = False

        if self.request.user.is_authenticated:
            context["is_favorite"] = CourseFavorite.objects.filter(
                user=self.request.user,
                course=self.object,
            ).exists()

        context["favorite_count"] = self.object.favorites.count()

        return context
```

5.5. URLs

`courses/urls.py` (añadir):

```
path(
    "favorites/",
    views.CourseFavoriteListView.as_view(),
    name="favorites",
),
path(
    "<int:pk>/favorite/",
    views.toggle_course_favorite,
    name="toggle_favorite",
),
```

5.6. Templates

`templates/courses/course_list.html` (nuevo completo)

```
{% extends "base.html" %}

{% block title %}
Cursos | Academy Hub
{% endblock %}

{% block content %}

<h1>Cursos</h1>

<form method="get" action="{% url 'courses:list' %}">

    <p>
        <input type="search"
               name="q"
               value="{{ request.GET.q }}"
               placeholder="Buscar cursos...">
    </p>

    <p>
        <select name="category">
            <option value="">Todas las categorías</option>
            {% for cat in categories %}
                <option value="{{ cat.slug }}"
                    {% if request.GET.category == cat.slug %}selected{% endif %}>
                    {{ cat.name }}
                </option>
            {% endfor %}
        </select>
    </p>

    <p>
        <select name="level">
            <option value="">Cualquier nivel</option>
```

```
            {% for value, label in levels %}
                <option value="{{ value }}"
                    {% if request.GET.level == value %}selected{% endif %}>
                    {{ label }}
                </option>
            {% endfor %}
        </select>
    </p>

    <p>
        <select name="modality">
            <option value="">Cualquier modalidad</option>
            {% for value, label in modalities %}
                <option value="{{ value }}"
                    {% if request.GET.modality == value %}selected{% endif %}>
                    {{ label }}
                </option>
            {% endfor %}
        </select>
    </p>

    <p>
        <input type="number" name="min_price" min="0" step="0.01"
               placeholder="Precio mínimo"
               value="{{ request.GET.min_price }}">

        <input type="number" name="max_price" min="0" step="0.01"
               placeholder="Precio máximo"
               value="{{ request.GET.max_price }}">
    </p>

    <p>
        <select name="ordering">
            <option value="newest" {% if request.GET.ordering == "newest" or not request.GET.
                Más recientes
            </option>
            <option value="price_asc" {% if request.GET.ordering == "price_asc" %}selected{%
                Precio: menor a mayor
            </option>
            <option value="price_desc" {% if request.GET.ordering == "price_desc" %}selected{
                Precio: mayor a menor
            </option>
            <option value="popular" {% if request.GET.ordering == "popular" %}selected{% endi
                Más vistos
            </option>
        </select>
    </p>

    <button type="submit">Buscar y filtrar</button>

    <a href="{% url 'courses:list' %}">Limpiar filtros</a>

</form>
```

```
{% if user.is_authenticated and user.is_academy %}
    <p>
        <a href="{% url 'courses:create' %}">Publicar curso</a>
        |
        <a href="{% url 'courses:favorites' %}">Mis favoritos</a>
    </p>
{% endif %}

{% for course in courses %}

    <article>

        {% if course.primary_image %}
            <img src="{{ course.primary_image.image.url }}"
                 alt="{{ course.title }}"
                 width="200">
        {% endif %}

        <h2>
            <a href="{{ course.get_absolute_url }}">
                {{ course.title }}
            </a>
        </h2>

        <p>Academia: {{ course.academy.name }}</p>
        <p>Categoría: {{ course.category.name }}</p>

        {% if course.teacher %}
            <p>Profesor: {{ course.teacher }}</p>
        {% endif %}

        <p>Nivel: {{ course.get_level_display }}</p>
        <p>Modalidad: {{ course.get_modality_display }}</p>

        {% if user.is_authenticated and course.academy.owner_id != user.pk %}
            <form method="post"
                  action="{% url 'courses:toggle_favorite' course.pk %}">
                {% csrf_token %}
                <input type="hidden" name="next" value="{{ request.get_full_path }}">
                {% if course.pk in favorite_course_ids %}
                    <button type="submit">Quitar de favoritos</button>
                {% else %}
                    <button type="submit">Añadir a favoritos</button>
                {% endif %}
            </form>
        {% endif %}

    </article>

    <hr>

{% empty %}
```

```
    <p>No hay cursos que coincidan con los filtros.</p>

{% endfor %}

{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?{{ querystring }}&page={{ page_obj.previous_page_number }}">
                Anterior
            </a>
        {% endif %}

        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>

        {% if page_obj.has_next %}
            <a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">
                Siguiente
            </a>
        {% endif %}
    </nav>
{% endif %}

{% endblock %}
```

templates/courses/course_favorite_list.html

```
{% extends "base.html" %}

{% block title %}
Mis cursos favoritos | Academy Hub
{% endblock %}

{% block content %}

<h1>Mis cursos favoritos</h1>

{% for favorite in favorites %}

    {% with course=favorite.course %}

        <article>

            {% if course.primary_image %}
                <img src="{{ course.primary_image.image.url }}"
                     alt="{{ course.title }}"
                     width="200">
            {% endif %}

            <h2>
                <a href="{{ course.get_absolute_url }}">
                    {{ course.title }}
                </a>
```

```
            </h2>

            <p>{{ course.academy.name }}</p>

        </article>

        <hr>

    {% endwith %}

{% empty %}

    <p>No tienes cursos favoritos.</p>

{% endfor %}

{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?page={{ page_obj.previous_page_number }}">Anterior</a>
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}

{% endblock %}
```

`templates/courses/course_detail.html` (añadir al final, antes de `{% endblock %}`)

```
{% if user.is_authenticated and course.academy.owner_id != user.pk %}
    <form method="post"
          action="{% url 'courses:toggle_favorite' course.pk %}">
        {% csrf_token %}
        <input type="hidden" name="next" value="{{ request.get_full_path }}">
        {% if is_favorite %}
            <button type="submit">Quitar de favoritos</button>
        {% else %}
            <button type="submit">Añadir a favoritos</button>
        {% endif %}
    </form>
{% endif %}

<p>{{ favorite_count }} usuarios han guardado este curso.</p>
```

`templates/courses/course_form.html` (añadir `enctype`)

```
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
```

```
    <button type="submit">Guardar</button>
</form>
```

5.7. Tests

`courses/tests.py` (añadir)

```
from decimal import Decimal

from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse

from academies.models import Academy

from .models import Category, Convocatoria, Course, CourseFavorite, CourseStatus

User = get_user_model()


class CourseSearchTests(TestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(
            name="Idiomas",
            slug="idiomas",
        )
        self.course1 = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2 intensivo",
            slug="ingles-b2",
            description="Curso intensivo de inglés",
            status=CourseStatus.PUBLISHED,
        )
        self.course2 = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Francés B1",
            slug="frances-b1",
            description="Curso de francés",
```

```
            status=CourseStatus.PUBLISHED,
        )
        Convocatoria.objects.create(
            course=self.course1,
            start_date="2026-07-01",
            price=Decimal("480.00"),
            total_places=15,
        )

    def test_search_by_title(self):
        response = self.client.get(
            reverse("courses:list"),
            {"q": "Inglés"},
        )
        self.assertContains(response, "Inglés B2 intensivo")
        self.assertNotContains(response, "Francés B1")

    def test_filter_by_category(self):
        response = self.client.get(
            reverse("courses:list"),
            {"category": "idiomas"},
        )
        self.assertEqual(response.status_code, 200)

    def test_filter_by_price_range(self):
        response = self.client.get(
            reverse("courses:list"),
            {"min_price": "400", "max_price": "600"},
        )
        self.assertContains(response, "Inglés B2 intensivo")

    def test_only_published_in_list(self):
        Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Curso borrador",
            slug="curso-borrador",
            description="...",
            status=CourseStatus.DRAFT,
        )
        response = self.client.get(reverse("courses:list"))
        self.assertContains(response, "Inglés B2 intensivo")
        self.assertNotContains(response, "Curso borrador")


class CourseFavoriteTests(TestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
```

```
        )
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(name="Idiomas", slug="idiomas")
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status=CourseStatus.PUBLISHED,
        )

    def test_student_can_add_favorite(self):
        self.client.force_login(self.student)
        self.client.post(
            reverse("courses:toggle_favorite", kwargs={"pk": self.course.pk}),
        )
        self.assertTrue(
            CourseFavorite.objects.filter(
                user=self.student,
                course=self.course,
            ).exists()
        )

    def test_student_can_remove_favorite(self):
        CourseFavorite.objects.create(user=self.student, course=self.course)
        self.client.force_login(self.student)
        self.client.post(
            reverse("courses:toggle_favorite", kwargs={"pk": self.course.pk}),
        )
        self.assertFalse(
            CourseFavorite.objects.filter(
                user=self.student,
                course=self.course,
            ).exists()
        )

    def test_favorite_requires_login(self):
        response = self.client.post(
            reverse("courses:toggle_favorite", kwargs={"pk": self.course.pk}),
        )
        self.assertEqual(response.status_code, 302)
```

```
    def test_owner_cannot_favorite_own_course(self):
        self.client.force_login(self.owner)
        self.client.post(
            reverse("courses:toggle_favorite", kwargs={"pk": self.course.pk}),
        )
        self.assertFalse(
            CourseFavorite.objects.filter(
                user=self.owner,
                course=self.course,
            ).exists()
        )


class CourseViewsCounterTests(TestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(name="Idiomas", slug="idiomas")
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status=CourseStatus.PUBLISHED,
        )

    def test_course_view_increments_counter(self):
        initial = self.course.views_count
        self.client.get(self.course.get_absolute_url())
        self.course.refresh_from_db()
        self.assertEqual(self.course.views_count, initial + 1)
```

---

6. `academies` — Academia como entidad pública

6.1. Modelo `AcademyFavorite`

academies/models.py

```
class AcademyFavorite(models.Model):
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="academy_favorites",
    )
    academy = models.ForeignKey(
        Academy,
        on_delete=models.CASCADE,
        related_name="favorites",
    )
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        constraints = [
            models.UniqueConstraint(
                fields=["user", "academy"],
                name="unique_user_academy_favorite",
            ),
        ]

    def __str__(self):
        return f"{self.user} - {self.academy}"
```

6.2. Añadir `views_count` a `Academy`

```
views_count = models.PositiveIntegerField(default=0)
```

6.3. Vistas

academies/views.py

```
from django.db.models import F, Q
from django.db import transaction
from django.shortcuts import get_object_or_404, redirect
from django.views.decorators.http import require_POST
from django.contrib.auth.decorators import login_required
from django.utils.http import url_has_allowed_host_and_scheme

from .models import Academy, AcademyFavorite, Teacher


class AcademyListView(ListView):
    model = Academy
    template_name = "academies/academy_list.html"
    context_object_name = "academies"
    paginate_by = 12

    def get_queryset(self):
```

```
        queryset = (
            Academy.objects
            .filter(is_active=True)
            .select_related("owner")
            .prefetch_related("courses")
        )

        query = self.request.GET.get("q", "").strip()
        if query:
            queryset = queryset.filter(
                Q(name__icontains=query)
                | Q(description__icontains=query)
                | Q(city__icontains=query)
            )

        city = self.request.GET.get("city")
        if city:
            queryset = queryset.filter(city__icontains=city)

        ordering_map = {
            "newest": "-created_at",
            "name_asc": "name",
            "name_desc": "-name",
            "popular": "-views_count",
        }
        ordering = self.request.GET.get("ordering", "newest")
        queryset = queryset.order_by(ordering_map.get(ordering, "-created_at"))

        return queryset

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)

        context["favorite_academy_ids"] = set()
        if self.request.user.is_authenticated:
            context["favorite_academy_ids"] = set(
                AcademyFavorite.objects
                .filter(user=self.request.user)
                .values_list("academy_id", flat=True)
            )

        query_params = self.request.GET.copy()
        query_params.pop("page", None)
        context["querystring"] = query_params.urlencode()

        return context


class AcademyDetailView(DetailView):
    model = Academy
    template_name = "academies/academy_detail.html"
    context_object_name = "academy"
```

```
    def get_object(self, queryset=None):
        queryset = queryset or self.get_queryset()
        academy = queryset.get(slug=self.kwargs["slug"])

        Academy.objects.filter(pk=academy.pk).update(
            views_count=F("views_count") + 1
        )
        academy.refresh_from_db(fields=["views_count"])
        return academy

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["is_favorite"] = False

        if self.request.user.is_authenticated:
            context["is_favorite"] = AcademyFavorite.objects.filter(
                user=self.request.user,
                academy=self.object,
            ).exists()

        context["favorite_count"] = self.object.favorites.count()

        return context


@login_required
@require_POST
def toggle_academy_favorite(request, slug):
    academy = get_object_or_404(Academy, slug=slug, is_active=True)

    next_url = request.POST.get("next", "")
    if not url_has_allowed_host_and_scheme(
        next_url,
        allowed_hosts={request.get_host()},
        require_https=request.is_secure(),
    ):
        next_url = reverse("academies:detail", kwargs={"slug": academy.slug})

    if academy.owner_id == request.user.pk:
        messages.warning(request, "No puedes guardar tu propia academia.")
        return redirect(next_url)

    with transaction.atomic():
        favorite, created = AcademyFavorite.objects.get_or_create(
            user=request.user,
            academy=academy,
        )
        if not created:
            favorite.delete()
            messages.info(request, "Academia eliminada de favoritos.")
        else:
            messages.success(request, "Academia añadida a favoritos.")
```

```
    return redirect(next_url)


class AcademyFavoriteListView(LoginRequiredMixin, ListView):
    model = AcademyFavorite
    template_name = "academies/academy_favorite_list.html"
    context_object_name = "favorites"
    paginate_by = 12

    def get_queryset(self):
        return (
            AcademyFavorite.objects
            .filter(user=self.request.user)
            .select_related("academy", "academy__owner")
        )
```

6.4. URLs

`academies/urls.py` (añadir)

```
path(
    "favorites/",
    views.AcademyFavoriteListView.as_view(),
    name="favorites",
),
path(
    "<slug:slug>/favorite/",
    views.toggle_academy_favorite,
    name="toggle_favorite",
),
```

6.5. Templates

`templates/academies/academy_list.html`

```
{% extends "base.html" %}

{% block title %}
Academias | Academy Hub
{% endblock %}

{% block content %}

<h1>Academias</h1>

<form method="get" action="{% url 'academies:list' %}">

    <p>
        <input type="search"
               name="q"
```

```
                value="{{ request.GET.q }}"
                placeholder="Buscar academias...">
    </p>

    <p>
        <input type="text"
                name="city"
                value="{{ request.GET.city }}"
                placeholder="Ciudad">
    </p>

    <p>
        <select name="ordering">
            <option value="newest" {% if request.GET.ordering == "newest" or not request.GET.
                Más recientes
            </option>
            <option value="name_asc" {% if request.GET.ordering == "name_asc" %}selected{% en
                Nombre A-Z
            </option>
            <option value="name_desc" {% if request.GET.ordering == "name_desc" %}selected{%
                Nombre Z-A
            </option>
            <option value="popular" {% if request.GET.ordering == "popular" %}selected{% endi
                Más vistas
            </option>
        </select>
    </p>

    <button type="submit">Buscar y filtrar</button>
    <a href="{% url 'academies:list' %}">Limpiar filtros</a>

</form>

{% for academy in academies %}

    <article>

        {% if academy.logo %}
            <img src="{{ academy.logo.url }}" alt="{{ academy.name }}" width="150">
        {% endif %}

        <h2>
            <a href="{% url 'academies:detail' academy.slug %}">
                {{ academy.name }}
            </a>
        </h2>

        <p>{{ academy.city }}</p>

        {% if user.is_authenticated and academy.owner_id != user.pk %}
            <form method="post"
                  action="{% url 'academies:toggle_favorite' academy.slug %}">
                {% csrf_token %}
```

```
                <input type="hidden" name="next" value="{{ request.get_full_path }}">
                {% if academy.pk in favorite_academy_ids %}
                    <button type="submit">Quitar de favoritos</button>
                {% else %}
                    <button type="submit">Añadir a favoritos</button>
                {% endif %}
            </form>
        {% endif %}

    </article>

    <hr>

{% empty %}

    <p>No hay academias que coincidan con los filtros.</p>

{% endfor %}

{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?{{ querystring }}&page={{ page_obj.previous_page_number }}">Anterior</a
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}

{% endblock %}
```

templates/academies/academy_favorite_list.html

```
{% extends "base.html" %}

{% block title %}
Mis academias favoritas | Academy Hub
{% endblock %}

{% block content %}

<h1>Mis academias favoritas</h1>

{% for favorite in favorites %}

    {% with academy=favorite.academy %}

        <article>

            {% if academy.logo %}
                <img src="{{ academy.logo.url }}" alt="{{ academy.name }}" width="150">
```

```
            {% endif %}

            <h2>
                <a href="{% url 'academies:detail' academy.slug %}">
                    {{ academy.name }}
                </a>
            </h2>

            <p>{{ academy.city }}</p>

        </article>

        <hr>

    {% endwith %}

{% empty %}

    <p>No tienes academias favoritas.</p>

{% endfor %}

{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?page={{ page_obj.previous_page_number }}">Anterior</a>
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}

{% endblock %}
```

`templates/academies/academy_detail.html` (añadir al final)

```
{% if user.is_authenticated and academy.owner_id != user.pk %}
    <form method="post"
          action="{% url 'academies:toggle_favorite' academy.slug %}">
        {% csrf_token %}
        <input type="hidden" name="next" value="{{ request.get_full_path }}">
        {% if is_favorite %}
            <button type="submit">Quitar de favoritos</button>
        {% else %}
            <button type="submit">Añadir a favoritos</button>
        {% endif %}
    </form>
{% endif %}

<p>{{ favorite_count }} usuarios han guardado esta academia.</p>
```

6.6. Tests

`academies/tests.py` (añadir)

```
class AcademySearchTests(TestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
            name="English Now",
            slug="english-now",
            city="Madrid",
        )
        Academy.objects.create(
            owner=self.owner,
            name="Code Academy",
            slug="code-academy",
            city="Barcelona",
        )

    def test_search_by_name(self):
        response = self.client.get(
            reverse("academies:list"),
            {"q": "English"},
        )
        self.assertContains(response, "English Now")
        self.assertNotContains(response, "Code Academy")

    def test_filter_by_city(self):
        response = self.client.get(
            reverse("academies:list"),
            {"city": "Barcelona"},
        )
        self.assertContains(response, "Code Academy")
        self.assertNotContains(response, "English Now")


class AcademyFavoriteTests(TestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
```

```
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
            name="English Now",
            slug="english-now",
        )

    def test_student_can_add_favorite(self):
        self.client.force_login(self.student)
        self.client.post(
            reverse("academies:toggle_favorite", kwargs={"slug": "english-now"}),
        )
        self.assertTrue(
            AcademyFavorite.objects.filter(
                user=self.student,
                academy=self.academy,
            ).exists()
        )

    def test_owner_cannot_favorite_own_academy(self):
        self.client.force_login(self.owner)
        self.client.post(
            reverse("academies:toggle_favorite", kwargs={"slug": "english-now"}),
        )
        self.assertFalse(
            AcademyFavorite.objects.filter(
                user=self.owner,
                academy=self.academy,
            ).exists()
        )
```

---

7. `reviews` — Reseñas con filtros y paginación

7.1. Vistas

`reviews/views.py` (ampliar)

```
class CourseReviewListView(ListView):
    model = CourseReview
    template_name = "reviews/course_review_list.html"
    context_object_name = "reviews"
    paginate_by = 20

    def get_queryset(self):
        queryset = (
```

```
            CourseReview.objects
            .filter(is_published=True)
            .select_related("student", "course", "course__academy")
        )

        course_id = self.request.GET.get("course")
        if course_id:
            queryset = queryset.filter(course_id=course_id)

        rating = self.request.GET.get("rating")
        if rating:
            queryset = queryset.filter(rating=rating)

        ordering = self.request.GET.get("ordering", "newest")
        if ordering == "oldest":
            queryset = queryset.order_by("created_at")
        elif ordering == "best":
            queryset = queryset.order_by("-rating", "-created_at")
        else:
            queryset = queryset.order_by("-created_at")

        return queryset

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        query_params = self.request.GET.copy()
        query_params.pop("page", None)
        context["querystring"] = query_params.urlencode()
        return context
```

7.2. URLs

`reviews/urls.py` (añadir)

```
path(
    "course-reviews/",
    views.CourseReviewListView.as_view(),
    name="course_review_list",
),
```

7.3. Templates

`templates/reviews/course_review_list.html`

```
{% extends "base.html" %}

{% block title %}
Reseñas de cursos | Academy Hub
{% endblock %}
```

```
{% block content %}

<h1>Reseñas de cursos</h1>

<form method="get">

    <p>
        <select name="rating">
            <option value="">Todas las puntuaciones</option>
            <option value="5" {% if request.GET.rating == "5" %}selected{% endif %}>5 estrell
            <option value="4" {% if request.GET.rating == "4" %}selected{% endif %}>4 estrell
            <option value="3" {% if request.GET.rating == "3" %}selected{% endif %}>3 estrell
            <option value="2" {% if request.GET.rating == "2" %}selected{% endif %}>2 estrell
            <option value="1" {% if request.GET.rating == "1" %}selected{% endif %}>1 estrell
        </select>
    </p>

    <p>
        <select name="ordering">
            <option value="newest" {% if request.GET.ordering == "newest" or not request.GET.
                Más recientes
            </option>
            <option value="oldest" {% if request.GET.ordering == "oldest" %}selected{% endif
                Más antiguas
            </option>
            <option value="best" {% if request.GET.ordering == "best" %}selected{% endif %}>
                Mejor puntuación
            </option>
        </select>
    </p>

    <button type="submit">Filtrar</button>
    <a href="{% url 'reviews:course_review_list' %}">Limpiar</a>

</form>

{% for review in reviews %}

    <article>

        <p>Curso: {{ review.course.title }}</p>
        <p>Alumno: {{ review.student.username }}</p>
        <p>Puntuación: {{ review.rating }}/5</p>
        <p>{{ review.comment }}</p>

    </article>

    <hr>

{% empty %}

    <p>No hay reseñas que coincidan con los filtros.</p>
```

```
{% endfor %}

{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?{{ querystring }}&page={{ page_obj.previous_page_number }}">Anterior</a
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}

{% endblock %}
```

7.4. Tests

`reviews/tests.py` (añadir)

```
class CourseReviewListTests(TestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(name="Idiomas", slug="idiomas")
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status=CourseStatus.PUBLISHED,
        )
        CourseReview.objects.create(
            student=self.student,
            course=self.course,
```

```
            rating=5,
            comment="Excelente",
        )

    def test_review_list_is_public(self):
        response = self.client.get(reverse("reviews:course_review_list"))
        self.assertEqual(response.status_code, 200)

    def test_filter_by_rating(self):
        response = self.client.get(
            reverse("reviews:course_review_list"),
            {"rating": "5"},
        )
        self.assertContains(response, "Excelente")
```

---

8. `level_tests` — Pruebas de nivel con filtros

8.1. Vistas

`level_tests/views.py` (ampliar `LevelTestListView`)

```
class LevelTestListView(ListView):
    model = LevelTest
    template_name = "level_tests/level_test_list.html"
    context_object_name = "level_tests"
    paginate_by = 12

    def get_queryset(self):
        queryset = (
            LevelTest.objects
            .filter(is_active=True)
            .select_related("course", "course__academy")
        )

        query = self.request.GET.get("q", "").strip()
        if query:
            queryset = queryset.filter(
                Q(title__icontains=query) | Q(description__icontains=query)
            )

        course_id = self.request.GET.get("course")
        if course_id:
            queryset = queryset.filter(course_id=course_id)

        return queryset

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        query_params = self.request.GET.copy()
        query_params.pop("page", None)
```

```
        context["querystring"] = query_params.urlencode()
        return context
```

8.2. Templates

`templates/level_tests/level_test_list.html` (añadir formulario de filtro arriba y paginación abajo)

```
<form method="get">
    <p>
        <input type="search" name="q" value="{{ request.GET.q }}" placeholder="Buscar pruebas
    </p>
    <button type="submit">Buscar</button>
</form>

{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?{{ querystring }}&page={{ page_obj.previous_page_number }}">Anterior</a
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}
```

8.3. Tests

`level_tests/tests.py` (añadir)

```
class LevelTestSearchTests(TestCase):

    def test_search_by_title(self):
        response = self.client.get(
            reverse("level_tests:list"),
            {"q": "inglés"},
        )
        self.assertEqual(response.status_code, 200)
```

---

9. `accounts` — Listado de usuarios

9.1. Vistas

`accounts/views.py` (ampliar `UserListView`)

```
class UserListView(StaffRequiredMixin, ListView):
    model = User
```

```
    template_name = "accounts/user_list.html"
    context_object_name = "users"
    paginate_by = 20

    def get_queryset(self):
        queryset = User.objects.all().order_by("-created_at")

        query = self.request.GET.get("q", "").strip()
        if query:
            queryset = queryset.filter(
                Q(username__icontains=query)
                | Q(email__icontains=query)
                | Q(first_name__icontains=query)
                | Q(last_name__icontains=query)
            )

        role = self.request.GET.get("role")
        if role == "academy":
            queryset = queryset.filter(is_academy=True)
        elif role == "student":
            queryset = queryset.filter(is_student=True)
        elif role == "staff":
            queryset = queryset.filter(is_staff=True)

        return queryset

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        query_params = self.request.GET.copy()
        query_params.pop("page", None)
        context["querystring"] = query_params.urlencode()
        return context
```

9.2. Templates

`templates/accounts/user_list.html` (ampliar)

```
<form method="get">
    <p>
        <input type="search" name="q" value="{{ request.GET.q }}" placeholder="Buscar usuario
    </p>
    <p>
        <select name="role">
            <option value="">Todos los roles</option>
            <option value="academy" {% if request.GET.role == "academy" %}selected{% endif %}
            <option value="student" {% if request.GET.role == "student" %}selected{% endif %}
            <option value="staff" {% if request.GET.role == "staff" %}selected{% endif %}>Sta
        </select>
    </p>
    <button type="submit">Filtrar</button>
</form>
```

```
{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?{{ querystring }}&page={{ page_obj.previous_page_number }}">Anterior</a
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}
```

---

10. `enrollments` — Listado con filtros por estado

10.1. Vistas

`enrollments/views.py` (ampliar las listas)

```
class EnrollmentListView(LoginRequiredMixin, ListView):
    model = Enrollment
    template_name = "enrollments/my_enrollment_list.html"
    context_object_name = "enrollments"
    paginate_by = 10

    def get_queryset(self):
        queryset = (
            Enrollment.objects
            .filter(student=self.request.user)
            .select_related("convocatoria", "convocatoria__course")
            .order_by("-created_at")
        )

        status = self.request.GET.get("status")
        if status:
            queryset = queryset.filter(status=status)

        return queryset

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["statuses"] = EnrollmentStatus.choices
        query_params = self.request.GET.copy()
        query_params.pop("page", None)
        context["querystring"] = query_params.urlencode()
        return context
```

10.2. Templates

`templates/enrollments/my_enrollment_list.html` (añadir filtro y paginación)

```
<form method="get">
    <select name="status">
        <option value="">Todos los estados</option>
        {% for value, label in statuses %}
            <option value="{{ value }}" {% if request.GET.status == value %}selected{% endif
                {{ label }}
            </option>
        {% endfor %}
    </select>
    <button type="submit">Filtrar</button>
</form>

{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?{{ querystring }}&page={{ page_obj.previous_page_number }}">Anterior</a
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}
```

---

11. `payments` — Listado con filtros por estado

11.1. Vistas

`payments/views.py` (ampliar `PaymentListView`)

```
class PaymentListView(LoginRequiredMixin, ListView):
    model = Payment
    template_name = "payments/payment_list.html"
    context_object_name = "payments"
    paginate_by = 20

    def get_queryset(self):
        user = self.request.user
        if user.is_academy:
            queryset = Payment.objects.filter(
                enrollment__convocatoria__course__academy__owner=user
            )
        else:
            queryset = Payment.objects.filter(enrollment__student=user)

        queryset = queryset.select_related(
            "enrollment",
            "enrollment__student",
            "enrollment__convocatoria__course",
```

```
            "discount",
        )

        status = self.request.GET.get("status")
        if status:
            queryset = queryset.filter(status=status)

        return queryset.order_by("-created_at")

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["statuses"] = Payment.Status.choices
        query_params = self.request.GET.copy()
        query_params.pop("page", None)
        context["querystring"] = query_params.urlencode()
        return context
```

11.2. Templates

`templates/payments/payment_list.html` (añadir filtro y paginación)

```
<form method="get">
    <select name="status">
        <option value="">Todos los estados</option>
        {% for value, label in statuses %}
            <option value="{{ value }}" {% if request.GET.status == value %}selected{% endif
                {{ label }}
            </option>
        {% endfor %}
    </select>
    <button type="submit">Filtrar</button>
</form>

{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?{{ querystring }}&page={{ page_obj.previous_page_number }}">Anterior</a
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}
```

---

12. `notifications` — Paginación y filtros

12.1. Vistas

`notifications/views.py` (ampliar)

```
class NotificationListView(LoginRequiredMixin, ListView):
    model = Notification
    template_name = "notifications/notification_list.html"
    context_object_name = "notifications"
    paginate_by = 20

    def get_queryset(self):
        queryset = Notification.objects.filter(user=self.request.user)

        is_read = self.request.GET.get("is_read")
        if is_read == "1":
            queryset = queryset.filter(is_read=True)
        elif is_read == "0":
            queryset = queryset.filter(is_read=False)

        return queryset

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        query_params = self.request.GET.copy()
        query_params.pop("page", None)
        context["querystring"] = query_params.urlencode()
        return context
```

12.2. Templates

`templates/notifications/notification_list.html` (añadir filtro y paginación)

```
<form method="get">
    <select name="is_read">
        <option value="">Todas</option>
        <option value="0" {% if request.GET.is_read == "0" %}selected{% endif %}>No leídas</o
        <option value="1" {% if request.GET.is_read == "1" %}selected{% endif %}>Leídas</opti
    </select>
    <button type="submit">Filtrar</button>
</form>

{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?{{ querystring }}&page={{ page_obj.previous_page_number }}">Anterior</a
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}
```

13. `evaluations` — Listado con paginación

13.1. Vistas

`evaluations/views.py` (ampliar)

```
class EvaluationListView(AcademyRequiredMixin, ListView):
    model = Evaluation
    template_name = "evaluations/evaluation_list.html"
    context_object_name = "evaluations"
    paginate_by = 20

    def get_queryset(self):
        return (
            Evaluation.objects
            .filter(enrollment__convocatoria__course__academy__owner=self.request.user)
            .select_related("enrollment", "enrollment__student")
            .order_by("-created_at")
        )
```

13.2. Templates

Añadir paginación estándar al final de `evaluation_list.html`.

---

14. `talent` — Competencias con filtros

14.1. Vistas

`talent/views.py` (ampliar)

```
class CompetenceListView(StaffRequiredMixin, ListView):
    model = Competence
    template_name = "talent/competence_list.html"
    context_object_name = "competences"
    paginate_by = 20

    def get_queryset(self):
        queryset = Competence.objects.filter(is_active=True).select_related("category")

        query = self.request.GET.get("q", "").strip()
        if query:
            queryset = queryset.filter(name__icontains=query)

        category_id = self.request.GET.get("category")
        if category_id:
            queryset = queryset.filter(category_id=category_id)
```

```
        return queryset.order_by("name")

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["categories"] = Category.objects.filter(is_active=True)
        query_params = self.request.GET.copy()
        query_params.pop("page", None)
        context["querystring"] = query_params.urlencode()
        return context
```

14.2. Templates

`templates/talent/competence_list.html` (añadir filtros y paginación)

```
<form method="get">
    <input type="search" name="q" value="{{ request.GET.q }}" placeholder="Buscar competencia
    <select name="category">
        <option value="">Todas las categorías</option>
        {% for cat in categories %}
            <option value="{{ cat.pk }}" {% if request.GET.category == cat.pk|stringformat:"s
                {{ cat.name }}
            </option>
        {% endfor %}
    </select>
    <button type="submit">Filtrar</button>
</form>

{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?{{ querystring }}&page={{ page_obj.previous_page_number }}">Anterior</a
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}
```

Nota: en este caso `cat.pk|stringformat:"s"` funciona porque va dentro del `value` de una opción, no dentro de un `{% if %}` con filtro directo. Si diera problema, aplica el patrón `cat.selected` ya visto en `search`.

---

15. `search` — Buscador unificado

15.1. Vistas

`search/views.py` (ampliar)

```
class GlobalSearchView(ListView):
    template_name = "search/global_search.html"
    context_object_name = "results"
    paginate_by = 20

    def get_queryset(self):
        query = self.request.GET.get("q", "").strip()
        if not query:
            return []

        results = list(
            Course.objects
            .filter(
                Q(title__icontains=query) | Q(description__icontains=query),
                status=CourseStatus.PUBLISHED,
            )
            .select_related("academy", "category")[:50]
        ) + list(
            Academy.objects
            .filter(
                Q(name__icontains=query) | Q(description__icontains=query),
                is_active=True,
            )
            .select_related("owner")[:50]
        ) + list(
            Teacher.objects
            .filter(
                Q(first_name__icontains=query)
                | Q(last_name__icontains=query)
                | Q(specialty__icontains=query),
                is_active=True,
            )
            .select_related("academy")[:50]
        )

        return results

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["current_q"] = self.request.GET.get("q", "")
        return context
```

15.2. URLs

`search/urls.py` (añadir)

```
path("global/", views.GlobalSearchView.as_view(), name="global"),
```

15.3. Templates

templates/search/global_search.html

```
{% extends "base.html" %}

{% block title %}
Buscador global | Academy Hub
{% endblock %}

{% block content %}

<h1>Buscar en Academy Hub</h1>

<form method="get">
    <input type="search" name="q" value="{{ current_q }}" placeholder="Buscar cursos, academi
    <button type="submit">Buscar</button>
</form>

{% for item in results %}

    <article>

        {% if item.title %}
            <h2>
                <a href="{{ item.get_absolute_url }}">
                    {{ item.title }}
                </a>
            </h2>
            <p>Curso</p>
        {% elif item.name %}
            <h2>
                <a href="{% url 'academies:detail' item.slug %}">
                    {{ item.name }}
                </a>
            </h2>
            <p>Academia</p>
        {% else %}
            <h2>{{ item }}</h2>
            <p>Profesor en {{ item.academy.name }}</p>
        {% endif %}

    </article>

    <hr>

{% empty %}

    {% if current_q %}
        <p>No se encontraron resultados para "{{ current_q }}".</p>
    {% else %}
        <p>Introduce un término de búsqueda.</p>
    {% endif %}

{% endfor %}
```

```
{% endblock %}
```

---

16. `core` — ContactMessage con filtros

16.1. Vistas

`core/views.py` (ampliar `ContactMessageListView`)

```
class ContactMessageListView(StaffRequiredMixin, ListView):
    model = ContactMessage
    template_name = "core/contact_message_list.html"
    context_object_name = "messages_list"
    paginate_by = 20

    def get_queryset(self):
        queryset = ContactMessage.objects.all().order_by("-created_at")

        is_read = self.request.GET.get("is_read")
        if is_read == "1":
            queryset = queryset.filter(is_read=True)
        elif is_read == "0":
            queryset = queryset.filter(is_read=False)

        return queryset

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        query_params = self.request.GET.copy()
        query_params.pop("page", None)
        context["querystring"] = query_params.urlencode()
        return context
```

16.2. Templates

`templates/core/contact_message_list.html` (añadir filtros y paginación)

```
<form method="get">
    <select name="is_read">
        <option value="">Todos</option>
        <option value="0" {% if request.GET.is_read == "0" %}selected{% endif %}>No leídos</o
        <option value="1" {% if request.GET.is_read == "1" %}selected{% endif %}>Leídos</opti
    </select>
    <button type="submit">Filtrar</button>
</form>

{% if is_paginated %}
    <nav>
```

```
        {% if page_obj.has_previous %}
            <a href="?{{ querystring }}&page={{ page_obj.previous_page_number }}">Anterior</a
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}
```

---

17. `dashboard` — Contadores y métricas

17.1. Vistas

`dashboard/views.py` (ampliar el `get_context_data`)

```
from django.db.models import Count, Avg

from courses.models import Course, CourseStatus
from enrollments.models import Enrollment
from payments.models import Payment
from reviews.models import CourseReview


class DashboardView(LoginRequiredMixin, TemplateView):
    def get_template_names(self):
        user = self.request.user
        if user.is_staff:
            return ["dashboard/staff_dashboard.html"]
        if user.is_academy:
            return ["dashboard/academy_dashboard.html"]
        return ["dashboard/student_dashboard.html"]

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        user = self.request.user

        if user.is_staff:
            context["total_courses"] = Course.objects.count()
            context["total_enrollments"] = Enrollment.objects.count()
            context["total_reviews"] = CourseReview.objects.count()
            context["avg_rating"] = (
                CourseReview.objects.aggregate(avg=Avg("rating"))["avg"] or 0
            )

        elif user.is_academy:
            context["my_courses"] = Course.objects.filter(academy__owner=user)
            context["my_enrollments"] = Enrollment.objects.filter(
                convocatoria__course__academy__owner=user
            )
```

```
            context["my_courses_count"] = context["my_courses"].count()
            context["published_courses_count"] = context["my_courses"].filter(
                status=CourseStatus.PUBLISHED
            ).count()
            context["enrollments_count"] = context["my_enrollments"].count()

        else:
            context["my_enrollments"] = Enrollment.objects.filter(student=user)
            context["my_payments"] = Payment.objects.filter(enrollment__student=user)
            context["my_reviews"] = CourseReview.objects.filter(student=user)
            context["enrollments_count"] = context["my_enrollments"].count()

        return context
```

---

18. Optimización ORM transversal

Reglas aplicadas en todas las vistas:

- `select_related` para cada `ForeignKey` mostrado en listados.
- `prefetch_related` para `1:N` y `M:N`.
- `distinct()` cuando se filtran relaciones `1:N` (cursos filtrados por convocatoria, por ejemplo).

Ejemplo en `CourseListView`:

```
queryset = queryset.distinct()
```

Ejemplo en `CourseFavoriteListView`:

```
.prefetch_related("course__images")
```

---

19. Paginación con querystring

Patrón aplicado en todas las vistas paginadas:

En la vista:

```
query_params = self.request.GET.copy()
query_params.pop("page", None)
context["querystring"] = query_params.urlencode()
```

En el template:

```
{% if is_paginated %}
    <nav>
        {% if page_obj.has_previous %}
            <a href="?{{ querystring }}&page={{ page_obj.previous_page_number }}">Anterior</a
```

```
        {% endif %}
        <span>Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}</span>
        {% if page_obj.has_next %}
            <a href="?{{ querystring }}&page={{ page_obj.next_page_number }}">Siguiente</a>
        {% endif %}
    </nav>
{% endif %}
```

---

20. Tests transversales

Además de los tests ya añadidos por app, se añaden los siguientes casos críticos:

- Búsqueda por texto en `courses`, `academies`, `level_tests`.
- Filtro por categoría, nivel, modalidad, ciudad.
- Filtro por rango de precio en cursos.
- Favoritos en cursos y academias.
- Un usuario no puede guardar su propio curso/academia.
- Favorito requiere login.
- Contador de vistas incrementa en cursos y academias.
- `distinct()` evita duplicados en cursos filtrados por convocatoria.

Ejecutar:

```
python manage.py test
```

Resultado esperado:

```
OK
```

---

21. Checklist global

Imágenes

- [x] Campo múltiple reutilizable en `core/forms.py`.
- [x] Validación de máximo 8 imágenes.
- [x] Validación de máximo 5 MB por imagen.
- [x] `enctype="multipart/form-data"` en formularios con archivos.
- [x] Primera imagen marcada como principal.
- [x] Posiciones calculadas.

Favoritos

- [x] Modelo `CourseFavorite` en `courses`.

- [x] Modelo `AcademyFavorite` en `academies`.
- [x] `UniqueConstraint` en ambos.
- [x] Toggle con POST.
- [x] No permitir guardar lo propio.
- [x] Listados de favoritos.
- [x] Botón en detalle.

Búsqueda

- [x] Por título, descripción, objetivos, competencias, academia, categoría.
- [x] Uso de `Q`.
- [x] Buscador unificado en `search`.

Filtros

- [x] Categoría.
- [x] Nivel.
- [x] Modalidad.
- [x] Ciudad (academias).
- [x] Precio mínimo / máximo.
- [x] Estado (enrollments, payments).
- [x] Leído / no leído (notifications, contact).

Ordenación

- [x] Recientes.
- [x] Precio ascendente / descendente.
- [x] Más vistos.
- [x] Nombre A-Z / Z-A.
- [x] Mejor puntuación.

Paginación

- [x] En todos los listados.
- [x] Persistencia de filtros con `querystring`.

ORM

- [x] `select_related`.
- [x] `prefetch_related`.
- [x] `distinct()`.
- [x] `F()` para contador de vistas.
- [x] `Q()` para búsqueda.

Tests

- [x] Búsqueda por título.
- [x] Filtro por categoría.
- [x] Filtro por rango de precio.
- [x] Añadir favorito.
- [x] Quitar favorito.
- [x] Favorito requiere login.
- [x] Propietario no puede guardar lo suyo.
- [x] Contador de visitas.

---

22. Registro de sesión

```
PROYECTO: Academy Hub
SESIÓN: 06
FASE: Desarrollo del núcleo de la plataforma
TEMA: Imágenes, favoritos, búsqueda, filtros y ORM avanzado
FECHA:
HORA DE ENTRADA:
HORA DE SALIDA:
DURACIÓN:

OBJETIVO:
Convertir Academy Hub en una plataforma navegable mediante imágenes múltiples,
favoritos, búsqueda, filtros, ordenación, paginación y optimización ORM.

TEORÍA:
- request.FILES
- Multiple uploads
- Q objects
- F expressions
- Query strings
- distinct()
- select_related / prefetch_related
- UniqueConstraint
- Paginación con querystring

FUNCIONALIDADES:
- Subida múltiple de imágenes (cursos y academias).
- Validación de imágenes.
- Favoritos de cursos y academias.
- Búsqueda por texto en cursos, academias y pruebas de nivel.
- Filtros: categoría, nivel, modalidad, ciudad, precio, estado, leído.
- Ordenación: recientes, precio, más vistos, nombre, puntuación.
- Paginación con persistencia de filtros.
- Contador de visitas.
- Buscador unificado.

APPS MODIFICADAS:
```

```
courses
academies
reviews
level_tests
accounts
enrollments
payments
notifications
evaluations
talent
search
core
dashboard

TESTS:
- búsqueda por título
- filtro por categoría
- filtro por rango de precio
- añadir favorito
- quitar favorito
- favorito requiere login
- propietario no puede guardar lo suyo
- contador de visitas

COMANDOS:
python manage.py makemigrations
python manage.py migrate
python manage.py check
python manage.py test

PROBLEMAS:
SOLUCIONES:

COMMIT:
feat: add images, favorites, search and filters across apps

ESTADO:
[ ] Pendiente
[ ] En progreso
[ ] Completada

PRÓXIMA SESIÓN:
PostgreSQL, ORM avanzado, constraints, índices, transacciones y
optimización de consultas.
```

---

23. Commit recomendado

```
git status
git add .
git commit -m "feat: add images, favorites, search and filters across apps"
```

24. Resultado final

Antes:

```
Academy Hub
    CRUD de todas las entidades
```

Ahora:

```
Academy Hub

Visitante
    puede ver:
        - cursos
        - academias
        - profesores
        - pruebas de nivel
    puede buscar y filtrar

Alumno
    puede:
        - guardar cursos y academias en favoritos
        - ver sus favoritos
        - seguir con sus inscripciones

Academia
    puede:
        - subir múltiples imágenes por curso
        - subir logo de academia
        - ver métricas en su panel

Staff
    puede:
        - ver mensajes con filtros
        - gestionar competencias con filtros
        - ver usuarios con filtros
```

Con esta sesión, Academy Hub se comporta como una plataforma real, con buscador, filtros y favoritos en todas las entidades que lo requieren.